Farmers Market Database: 200 SQL Interview Questions (up to, but not including, JOINs)

Topics: SELECT & derived columns · WHERE / AND / OR / NOT · IN / NOT IN · BETWEEN · LIKE · CONCAT / CONCAT_WS · INSTR / LOCATE · LEFT / RIGHT / SUBSTRING / SUBSTRING_INDEX · TRIM / LENGTH / CHAR_LENGTH / REPLACE / REVERSE · NULL handling (IS NULL, COALESCE, IFNULL, NULLIF) · CASE and SUM(CASE) · numeric and date functions · aggregates and DISTINCT · GROUP BY / HAVING · ORDER BY / LIMIT / OFFSET · order of execution · subqueries · statistics · interview logic

SQL flavour: MySQL 8.0 (MySQL Workbench). The answers use only core SQL (no CTEs), so almost all of them also run on the MySQL 5.6 editor on InterviewBit. Where 5.6 behaves differently, the answer says so.

Rules

- **No JOINs.** Each question needs only one table. When a question uses two tables, connect them with a **subquery** (`IN` , `EXISTS` , scalar or correlated).
- Give every derived column a clear alias.
- For **"What will this return?"** questions, work out the answer on paper first, then run the query to check it.
- The answers are in the separate file *Farmers_Market_200_SQL_Questions_with_Answers*.

Setup: in Workbench, run `farmers_market_setup.sql` . It creates the `farmers_market` database and loads all 9 tables from the CSVs in `csv/` .

> **About the data:** it was copied from the InterviewBit output, which cuts long results short. `customer_purchases` stops at 2019-09-04 for product 2, and `market_date_info` stops at 2020-07-25. If you run these queries on InterviewBit's full data, your numbers will differ from the ones quoted in the answers.

Schema Reference and Quirks

Table	Columns	Quirks worth knowing
`market_date_info`	market_date, market_day, market_week, market_year, market_start_time, market_end_time, special_notes, market_season, market_min_temp, market_max_temp, market_rain_flag, market_snow_flag	`market_week`, `market_year` and both temperature columns are **VARCHAR**. Start and end times are text such as `'8:00 AM'`. Temperatures are NULL on most days. `special_notes` is always NULL.
`booth`	booth_number, booth_price_level (A/B/C), booth_description, booth_type	Several descriptions contain a **double space** before the `-`. Booths 8 and 9 have **identical** descriptions.
`vendor`	vendor_id, vendor_name, vendor_type, vendor_owner_first_name, vendor_owner_last_name	'Hernández' has an accented **á**. Apostrophes are missing ('Annies Pies', 'Chriss ...').
`customer`	customer_id, customer_first_name, customer_last_name, customer_zip	`customer_zip` is VARCHAR. Two customers have the last name Diaz.
`product_category`	product_category_id, product_category_name	Names contain `&` and `( )`.
`product`	product_id, product_name, product_size, product_category_id, product_qty_type	Size is NULL for product 14 and a single **space** for product 15. Qty type is NULL for products 14 and 15. Some sizes are bare numbers ('10', '6'). There's a typo: 'Resuable'.
`vendor_booth_assignments`	vendor_id, booth_number, market_date	In summer 2019, vendor 4 **and** vendor 7 are both assigned to booth 11 on the same dates.
`vendor_inventory`	market_date, quantity, vendor_id, product_id, original_price	Only vendors 7 and 8 have inventory.
`customer_purchases`	product_id, vendor_id, market_date, customer_id, quantity, cost_to_customer_per_qty, transaction_time	`transaction_time` is TIME. On 2019-07-13, two customers paid **3.00** for product 2 instead of the usual 3.49.

Wednesday markets run 4:00–7:00 PM and Saturday markets run 8:00 AM–2:00 PM. Prices are in $, weights in lbs and temperatures in °F.

Section 1: SELECT, Derived Columns & Aliases

Q1. Finance wants a line-level sales report. From `customer_purchases`, show `market_date`, `customer_id`, `product_id`, `quantity`, `cost_to_customer_per_qty` and a column `line_total` (quantity × price) rounded to 2 decimals.

Q2. Assume a 5.3% sales tax. For every row in `customer_purchases`, show `line_total`, `tax_amount` and `amount_payable`, each rounded to 2 decimals. Then explain why adding the rounded `line_total` and rounded `tax_amount` can differ from the rounded `amount_payable` by a cent.

Q3. The manager wants to know how much stock (by value) vendors bring. From `vendor_inventory`, show `market_date`, `vendor_id`, `product_id`, `quantity`, `original_price` and `stock_value`. Return only the 10 highest-value rows.

Q4. From `market_date_info`, show `market_date`, the min and max temperature, `temp_swing` (max − min) and `avg_temp_c` (the average of min and max, converted to Celsius with `(F − 32) × 5/9` and rounded to 1 decimal). Both temperature columns are VARCHAR, so does the arithmetic still work? What shows up on days with no reading?

Q5. Products 1 and 2 are sold by the pound. For each purchase in `customer_purchases`, show `qty_kg` (1 lb = 0.4536 kg, rounded to 3 decimals) and `price_per_kg` (rounded to 2 decimals).

Q6. What will this return?

```
SELECT 7/2 AS a, 7 DIV 2 AS b, 7 % 2 AS c, -7 DIV 2 AS d, -7 % 2 AS e, 1 + '1 lb' AS f;
```

Q7. What happens when you run this query? Fix it.

```
SELECT quantity * cost_to_customer_per_qty AS line_total
FROM customer_purchases
WHERE line_total > 30;
```

Q8. Loyalty points equal the whole dollars spent on a line. Cents are dropped, not rounded, so a $39.91 line earns 39 points. From `customer_purchases`, show `line_total` and `reward_points`.

Q9. The market is planning a 12.5% markup, with the new price rounded **up** to the next nickel ($0.05). For example, 6.99 × 1.125 = 7.86375 becomes 7.90. From `vendor_inventory`, show the distinct `product_id`, `original_price` and `new_price`.

Q10. List every vendor with two constant columns: `market_name` = 'Harrisonburg Farmers Market' and `onboarding_year` = 2019, plus the vendor name.

Q11. What will this return?

```
SELECT product_id, product_id + 0.5, product_id / 0, product_name + 0
FROM product
WHERE product_id = 7;
```

Q12. A printed receipt needs headers with spaces in them. From `customer_purchases`, show **"Customer ID"**, **"Market Day"** (the weekday name of `market_date`) and **"Amount ($)"** (the rounded line total). Sort by "Amount ($)" from highest to lowest.

Section 2: WHERE, AND / OR / NOT

Q13. Find all Wednesday markets in 2019 that had rain. `market_year` is VARCHAR. Does `market_year = 2019` still work?

Q14. The manager asked for "Saturday markets with rain or snow" and got this query:

```
SELECT * FROM market_date_info
WHERE market_day = 'Saturday' AND market_rain_flag = 1 OR market_snow_flag = 1;
```

On this data the output happens to look right. Why is the query still wrong? Fix it.

Q15. Find the Jalapeno pepper purchases (product 2) that were sold at a price other than the regular 3.49.

Q16. Find the transactions made in the last 30 minutes before closing. Wednesday markets close at 7:00 PM and Saturday markets at 2:00 PM. Use only `customer_purchases`.

Q17. Products 1 and 2 are weighed, so a whole-number quantity such as exactly 2.00 lbs is unusual. Find the purchases whose quantity has no fractional part, without using LIKE.

Q18. Find the market days that were neither rainy nor snowy. Write it twice: once with `NOT (… OR …)` and once using De Morgan's law.

Q19. What does this return, and why isn't it 126 (128 rows minus the 2 days with a min temp of 50)?

```
SELECT COUNT(*) FROM market_date_info WHERE market_min_temp <> 50;
```

Q20. Find the customers in zip 22801 whose last name starts with a letter from A to M. Use comparison operators, not LIKE. Why does `customer_last_name <= 'M'` miss 'Messina'?

Q21. Find purchase lines worth more than $20 that were made before noon.

Q22. Find the market days where **exactly one** of rain or snow happened. Write it with `XOR`, then without it.

Q23. What does this return? Does it run, and is there any hidden cost?

```
SELECT * FROM customer WHERE customer_zip = 22801;
```

Q24. A colleague wanted the markets that start before noon and wrote this:

```
SELECT * FROM market_date_info WHERE market_start_time < '12:00 PM';
```

It returns 0 rows. Explain why, then fix it.

Q25. Find the products whose `product_qty_type` is **not** 'unit'. The products where the qty type is unknown should also appear. Why does `product_qty_type <> 'unit'` alone miss some rows?

Q26. The goal is product 2 purchases on 2019-07-13 by customer 2 or customer 10. Find the bug and fix it:

```
SELECT * FROM customer_purchases
WHERE market_date = '2019-07-13' AND customer_id = 2 OR customer_id = 10 AND product_id = 2;
```

Section 3: IN, NOT IN, BETWEEN

Q27. Find the purchases made by customers 1, 4 and 14 at Saturday markets.

Q28. Using `IN`, list the vendors whose type is 'Prepared Foods' or 'Eggs & Meats'.

Q29. List the booths that have **never** been assigned to a vendor. Use `booth` with a subquery on `vendor_booth_assignments`.

Q30. What will this return? Why? Fix it.

```
SELECT product_name FROM product WHERE product_qty_type NOT IN ('unit', NULL);
```

Q31. Using BETWEEN on `market_date`, find all purchases from July 2019. Then answer: if `market_date` were a DATETIME, why would `BETWEEN '2019-07-01' AND '2019-07-31'` lose most of July 31st?

Q32. Find the markets whose min temperature was between 40 and 55 °F. The column is VARCHAR. Is `BETWEEN '40' AND '55'` safe? When would it break?

Q33. What will this return?

```
SELECT COUNT(*) FROM customer_purchases WHERE quantity BETWEEN 5 AND 3;
```

Q34. Find the transactions made between 11:00 AM and 1:00 PM (inclusive) at Saturday markets.

Q35. Find the customers whose first name falls alphabetically from 'B' through 'D'. What happens to 'Deanna', 'Dawn' and 'Darrell' with `BETWEEN 'B' AND 'D'`? Fix the query.

Q36. Without a JOIN, find the products in the categories 'Fresh Fruits & Vegetables' or 'Eggs & Meat (Fresh or Frozen)'.

Q37. Find the vendors that have **never** brought inventory to the market.

Q38. Using `NOT BETWEEN`, find the purchases that were **not** made during the 2019 selling window ('2019-06-01' to '2019-10-30'). How many rows do you expect, and why?

Section 4: LIKE

Q39. Find all organic products. Does `LIKE '%organic%'` also match 'Organic' with a capital O? Why?

Q40. Find the products sold in a jar or a bag, meaning their names end with 'Jar' or 'Bag'.

Q41. Find the vendors whose name contains '&'. Then find the vendor types that contain a colon.

Q42. Using LIKE only, find the customers whose first name has exactly 3 letters.

Q43. Find the customers whose last name has 'a' as the **second** letter.

Q44. How would you search `product_name` for a literal underscore `_` or percent sign `%`? Write the query for underscore, and show two ways to escape it.

Q45. Find the vendors whose business name **begins with** the owner's first name (e.g. 'Marcos Peppers' for Marco).

Q46. Find the vendors whose business name contains the owner's **last name** anywhere.

Q47. Data-quality check: find the booths whose description contains a double space.

Q48. Marketing asks for the 'Red Potatoes' products. Compare `product_name = 'Red Potatoes'` with `product_name LIKE 'Red Potatoes%'`. How many rows does each return, and which fits the request?

Q49. Find the products whose size **starts with a digit** (e.g. '8 oz', '1.5 lbs', '10'). Write it without REGEXP, then with REGEXP.

Q50. Find the customers whose first and last names start with the same letter.

Section 5: CONCAT & Formatting

Q51. Build a display name such as **'Connor, Jane'** for every customer, sorted by last name, then first name.

Q52. Generate loyalty-card emails in the form first initial + last name + customer_id + '@fmarket.com', all lowercase. Customer 1 gets `jconnor1@fmarket.com`.

Q53. Build a product label such as **'Habanero Peppers - Organic (medium, lbs)'**. What happens to products 14 and 15? Rewrite it so that no label comes out NULL.

Q54. What will this return?

```
SELECT CONCAT('Booth-', 7), CONCAT('A', NULL, 'B'), CONCAT_WS('-', 'A', NULL, 'B'),
CONCAT_WS(NULL, 'A', 'B');
```

Q55. Build booth codes such as **'A-01'**, **'C-09'** and **'B-12'** from `booth_price_level` and a zero-padded `booth_number`.

Q56. Create a market-day label such as **'Sat, 02-Mar-2019 (Wk 09)'** from `market_date_info`.

Q57. Build an invoice line string for each purchase in this form: **'Cust 14 bought 0.99 lbs @ $6.99 = $6.92'**.

Q58. Mask customer last names for a public leaderboard. 'Connor' becomes **'C*'**: keep the first letter and replace the rest with stars.

Q59. Build a social-media handle from `vendor_name`: all lowercase, ' & ' replaced by '*and*' and the remaining spaces replaced by '_'. 'Chriss Sustainable Eggs & Meats' becomes

`chriss_sustainable_eggs_and_meats`.

Q60. What does `SELECT 'Eggs' || 'Meat';` return in MySQL, and why? How do you concatenate correctly?

Section 6: INSTR, LOCATE, LEFT, RIGHT, SUBSTRING, SUBSTRING_INDEX

Q61. Split `product_name` into `base_name` and `variant` at ' - '. For example, 'Habanero Peppers - Organic' becomes 'Habanero Peppers' and 'Organic'. When a product has no dash, `base_name` should be the full name and `variant` should be NULL. Use INSTR with LEFT and SUBSTRING, **not** SUBSTRING_INDEX.

Q62. Solve Q61 with SUBSTRING_INDEX. Why does `SUBSTRING_INDEX(product_name, ' - ', -1)` return a wrong variant for 'Eggs' or 'Sweet Corn'?

Q63. Extract the **first word** of every product name. Make sure one-word names such as 'Eggs' still work, then explain what `LEFT(name, INSTR(name, ' ') - 1)` returns for 'Eggs'.

Q64. Extract the **last word** of every product name in two ways: with SUBSTRING_INDEX, and with REVERSE + INSTR.

Q65. From `booth_description`, extract the table-size text after the dash (e.g. 'Small square table size'). Return NULL when there is no dash, and make sure the double spaces don't break the result.

Q66. Extract the first word of every booth description ('First', 'Second', 'Third', 'Back'), then count the booths for each word.

Q67. From `market_start_time` (e.g. '8:00 AM' or '4:00 PM'), extract the hour as a number and the AM/PM part **without hard-coding positions**. Remember that '12:00 PM' is longer than '8:00 AM'.

Q68. Without STR_TO_DATE, convert `market_start_time` and `market_end_time` to 24-hour numbers using LEFT/INSTR/RIGHT and CASE. Then compute the market's length in hours. Which date has an unusual length?

Q69. For sizes such as '8 oz', '1.5 lbs', '1 lb' and '1 dozen', split `product_size` into `size_value` (the text before the first space) and `size_unit` (the text after it). Only take rows whose size contains a space and starts with a digit.

Q70. What will this return?

```
SELECT LEFT('Apple Pie', 0), RIGHT('Apple Pie', 3), SUBSTRING('Apple Pie', 0, 3),
       SUBSTRING('Apple Pie', -3), SUBSTRING('Apple Pie', 7), MID('Apple Pie', 2, 3);
```

Q71. What will this return?

```
SELECT INSTR('Apple Pie', 'p'), LOCATE('p', 'Apple Pie'), LOCATE('p', 'Apple Pie', 4),
       INSTR('Apple Pie', 'z'), POSITION('Pie' IN 'Apple Pie');
```

Q72. For product names with at least two spaces, find the position of the **second** space.

Q73. For product names with **exactly three words**, extract the middle word. 'Cut Zinnias Bouquet' gives 'Zinnias'.

Q74. From `vendor_type`, extract the text before ':' when there is one ('Fresh Variety'). Otherwise return the full type.

Q75. Extract the hour and minute from `transaction_time` in two ways: with LEFT/SUBSTRING, and with HOUR()/MINUTE(). Why does the string version work on a TIME column?

Q76. Get the year and the month number from `market_date` using LEFT/SUBSTRING, then using YEAR()/MONTH(). Which approach is safer, and why?

Q77. Data check: find the products whose name contains ' - ' but whose part after the dash is not 'Organic', 'Jar' or 'Bag'.

Q78. Build a customer code such as **'JC-801-01'**: the initials, then the **last 3** digits of the zip, then the customer_id padded to 2 digits.

Section 7: TRIM, LENGTH, CHAR_LENGTH, REPLACE, REVERSE, UPPER/LOWER

Q79. What will this return?

```
SELECT LENGTH('Hernández'), CHAR_LENGTH('Hernández'), LENGTH(' '), CHAR_LENGTH(''),
LENGTH(NULL);
```

Q80. Using the difference between LENGTH and CHAR_LENGTH, find the vendors whose name or owner name contains non-ASCII (accented) characters.

Q81. Clean `booth_description` by collapsing double spaces into single spaces. Show the original length, the cleaned length and the cleaned text, only for the rows that changed.

Q82. Count the words in each product name. Count ' - ' as a separator, not as a word, so 'Habanero Peppers - Organic' has 3 words.

Q83. Count how many times the letter 'e' (upper or lower case) appears in each customer's full name. Why does `REPLACE(name, 'e', '')` alone undercount?

Q84. Find the customer first names that are palindromes, ignoring case.

Q85. What will this return?

```
SELECT TRIM('  Eggs  '), TRIM(LEADING '0' FROM '00120'), TRIM(TRAILING 's' FROM 'Peppers'),
       TRIM(BOTH '-' FROM '--Jar--'), LTRIM('  a '), RTRIM(' a  ');
```

Q86. Standardise `product_size` to sentence case: first letter uppercase and the rest lowercase ('medium' becomes 'Medium').

Q87. Find the product whose size is **blank but not NULL**. Why does `product_size = ''` find it on some servers but not others?

Q88. The search box is case-insensitive. Find the vendors with 'PIES' in their name. Then make the search **case-sensitive** and explain the new result.

Q89. Find the product(s) with the longest name, including ties.

Q90. Find the position of the **last** 'e' in each vendor name using REVERSE, and return 0 when there is none.

Section 8: NULL Handling

Q91. Find the market dates with no recorded temperature. Why does `WHERE market_min_temp = NULL` return nothing?

Q92. Show `market_date`, `market_min_temp` and `market_max_temp`, and display 'Not recorded' where a temperature is NULL.

Q93. What will this return?

```
SELECT NULL = NULL, NULL <=> NULL, NULL IS NULL, 1 + NULL, COALESCE(NULL, NULL, 'x'),
       IFNULL(NULL, 0), NULLIF(5, 5), NULLIF(5, 6), GREATEST(3, NULL);
```

Q94. Compare `COUNT(*)`, `COUNT(market_min_temp)` and `COUNT(special_notes)` in `market_date_info`, and explain each number.

Q95. Compute the average min temperature in three ways: `AVG(market_min_temp)`, `SUM(market_min_temp)/COUNT(*)` and `AVG(COALESCE(market_min_temp, 0))`. Which one is right, and why are the other two wrong?

Q96. Build a size label that shows 'Size not specified' when `product_size` is NULL **or** blank.

Q97. What does `SELECT 10/0;` return in MySQL, and how is that different from SQL Server or PostgreSQL? Rewrite "revenue per customer on each market date" so it can never divide by zero.

Q98. Find the products where size **or** qty type is missing, counting NULL, empty and space-only values as missing.

Q99. What will this return? The table has 23 products and 7 of them are 'lbs'.

```
SELECT COUNT(*) FROM product WHERE product_qty_type <> 'lbs';
```

Q100. Sort the markets by min temperature from lowest to highest. Where does MySQL put the NULLs? Rewrite the query so that NULLs come **last** and the temperatures sort as numbers.

Q101. Show 'Rain' or 'Dry' for each market day using `IF()`. How are `IFNULL` and `COALESCE` different?

Q102. What will this return?

```
SELECT COUNT(*), SUM(market_min_temp), AVG(market_min_temp), MAX(market_min_temp)
FROM market_date_info WHERE market_year = '2020';
```

Section 9: CASE and SUM(CASE)

Q103. Classify each market day by its max temperature: 'Cold' (< 50), 'Mild' (50–69), 'Warm' (≥ 70) and 'Unknown' (NULL).

Q104. Why does this CASE never return 'Warm'? Fix it.

```
CASE WHEN market_max_temp >= 50 THEN 'Mild'
     WHEN market_max_temp >= 70 THEN 'Warm'
     ELSE 'Cold' END
```

Q105. Label each purchase line as 'Small' (< $5), 'Medium' ($5–$20) or 'Large' (> $20). Show the line count and revenue for each label.

Q106. Pivot the table so that each product is one row, with the columns `sales_2019` and `sales_2020` for revenue.

Q107. For each customer, show `wed_lines` and `sat_lines` (the number of purchase lines on Wednesday and on Saturday).

Q108. What percentage of purchase lines were made before 12:00 noon? Write it with SUM(CASE), then with AVG of a boolean.

Q109. What will this return?

```
SELECT CASE WHEN NULL = NULL THEN 'equal' ELSE 'not equal' END,
       CASE NULL WHEN NULL THEN 'match' ELSE 'no match' END,
       CASE WHEN 1 = 2 THEN 'x' END;
```

Q110. Show the distinct market seasons in the business order Spring, then Summer/Early Fall, then Late Fall/Holiday. Do it with CASE in ORDER BY, then with FIELD().

Q111. Create a `weather` column with the values 'Rain & Snow', 'Rain', 'Snow' or 'Clear'.

Q112. For each season, find the percentage of market days that had rain.

Q113. Count the booths for each price level as **one row** with the columns `level_a`, `level_b` and `level_c`.

Q114. Flag each product 2 purchase as 'Discounted' or 'Full price' by comparing the price paid with the **highest** price ever paid for product 2. Use a scalar subquery inside CASE.

Q115. Use a single WHERE clause with CASE to return the Saturday purchases over $20 and the Wednesday purchases over $10.

Q116. Build a data-quality status for each product. The checks come in this order: 'Missing qty type' (NULL), 'Missing size' (NULL or blank), 'Size is a bare number' (the size contains only digits) and 'OK'. Which products get which status, and why does the order of the WHEN clauses matter?

Section 10: Numeric & Date Functions

Q117. What will this return?

```
SELECT ROUND(6.985, 2), ROUND(2.5), ROUND(-2.5), ROUND(1234.5, -2),
       TRUNCATE(6.999, 2), CEIL(4.01), FLOOR(-4.01), ROUND(2.5e0);
```

Q118. A cash-only stall rounds every bill to the nearest **quarter** ($0.25). Show `line_total` and `cash_total`.

Q119. For each market date, show the weekday, the month name, the quarter and the ISO week. Is the stored `market_week` the same as `WEEK(market_date)`? Which WEEK mode matches it?

Q120. As of 2020-10-10, how many days has it been since each customer's last purchase?

Q121. Find the purchases made in the first 7 days of any month.

Q122. Convert `market_start_time` and `market_end_time` to real TIME values with STR_TO_DATE, compute the market's length with TIMEDIFF, and sort the markets by their true start time.

Q123. For each transaction, compute the minutes since the market opened (16:00 on Wednesdays, 08:00 on Saturdays).

Q124. Using `DAYOFWEEK()`, find the purchases made on a Saturday. What number is Saturday, and which day is 1?

Q125. Show each purchase quantity as whole pounds plus leftover ounces (1 lb = 16 oz, rounded). For example, 2.18 lb is 2 lb 3 oz.

Q126. What will this return?

```
SELECT MOD(10, 3), 10 MOD 3, MOD(-10, 3), POWER(2, 10), SQRT(16), ABS(-3.5), SIGN(-3.5);
```

Section 11: Aggregates & DISTINCT

Q127. Build a one-row KPI summary of `customer_purchases`: purchase lines, distinct customers, total revenue, average line value, and the smallest and largest line value.

Q128. A **visit** is one (customer, market_date) pair. How many visits are there? Count them without a subquery.

Q129. What will this return?

```
SELECT COUNT(product_size), COUNT(DISTINCT product_size), COUNT(*) FROM product;
```

Q130. What was the true average price per pound paid for Jalapenos (product 2)? Compare `AVG(cost_to_customer_per_qty)` with the weighted average `SUM(q × p) / SUM(q)`, and say which one is correct.

Q131. Why does this query fail in MySQL 8? Fix it.

```
SELECT DISTINCT customer_id FROM customer_purchases ORDER BY market_date;
```

Q132. For vendor 7, how many distinct market days did it bring stock (`vendor_inventory`), and how many days did it record at least one sale (`customer_purchases`)?

Q133. For each vendor, show the total stock value brought across all dates and the total quantity brought.

Q134. What will this return, and why?

```
SELECT MAX(market_week), MIN(market_week), MAX(market_week + 0) FROM market_date_info;
```

Q135. Find the first and last market date on record, the number of calendar days between them and the number of market days held.

Q136. What will this return?

```
SELECT COUNT(*), SUM(quantity), AVG(quantity) FROM customer_purchases WHERE customer_id = 999;
```

Q137. How many distinct (product, price) combinations were sold? List them.

Q138. In one query, compute the average spend per **visit**: total revenue ÷ number of distinct (customer, date) pairs.

Section 12: GROUP BY & HAVING

Q139. Find the revenue for each market date, with the highest-revenue day first.

Q140. Show the revenue and distinct customers for each month, with the month labelled as **'Jul-2019'** and sorted in date order (not alphabetically).

Q141. Find the customers who spent more than $200 in total.

Q142. Find the customers who visited on at least 10 distinct dates but spent less than $200 in total.

Q143. For each customer, show the first purchase date, the last purchase date and the number of days between them (their tenure).

Q144. Find the customers who bought in 2019 but **never** in 2020, using a single GROUP BY with no subquery.

Q145. Find the customers who bought only **one** of the two products.

Q146. Find the cases where a customer bought the same product more than once on the same market date. Show the count and the transaction times (hint: GROUP_CONCAT).

Q147. Booth-conflict audit: find the (booth, date) pairs with **more than one** vendor assigned. How many are there?

Q148. Find the vendors assigned to **more than one booth** on the same date. List the booths as a comma-separated string.

Q149. Find the (year, week) groups whose market days fall in **different seasons**.

Q150. Every market week should have 2 market days (Wednesday and Saturday). Find the weeks that don't.

Q151. Find the revenue and line count for each hour of the day, split by market weekday. Which hour is busiest on Saturdays?

Q152. Why does this fail in MySQL? Rewrite it correctly and explain why WHERE is the better place for this filter.

```
SELECT customer_id, SUM(quantity * cost_to_customer_per_qty) AS spend
FROM customer_purchases
GROUP BY customer_id
HAVING market_date >= '2020-01-01';
```

Q153. This query ran on MySQL 5.6 but fails on MySQL 8 with error 1055. Explain why, and give two correct fixes.

```
SELECT customer_id, market_date, SUM(quantity) FROM customer_purchases GROUP BY customer_id;
```

Q154. Group the purchase lines into $5 buckets ('0-5', '5-10', ...) and show the count and revenue for each bucket, sorted by bucket.

Section 13: ORDER BY, LIMIT, OFFSET

Q155. Find the top 5 customers by total spend.

Q156. The customer list is shown 10 per page, sorted by last name and then first name. Write the query for **page 3**. Which customers appear, and how many?

Q157. Find the **second highest** distinct line total using LIMIT/OFFSET.

Q158. What do `LIMIT 1, 1`, `LIMIT 1 OFFSET 1` and `LIMIT 1, 3` each return?

Q159. Sort `market_date_info` rows by the market's **real** start time, then by date.

Q160. Find the top 3 purchases by quantity, including every row that ties with the 3rd-place value. Why can plain `LIMIT 3` be misleading?

Q161. Explain `GROUP BY 1 ORDER BY 2 DESC`. When is ordering by column position risky?

Q162. Why does `ORDER BY market_week` give 10, 11, ..., 52, 9? Sort the weeks correctly.

Section 14: Order of Execution

Q163. Write the **logical order** in which MySQL processes FROM, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY and LIMIT. Explain it using a query that finds 2020 customers with more than $100 of spend.

Q164. Why does `WHERE SUM(quantity) > 10` fail? Fix it.

Q165. Does this run in MySQL? Would it run in PostgreSQL or SQL Server?

```
SELECT customer_id, SUM(quantity * cost_to_customer_per_qty) AS spend
FROM customer_purchases GROUP BY customer_id HAVING spend > 200;
```

Q166. Does `GROUP BY yr` work when `yr` is an alias for `YEAR(market_date)`? Is it portable?

Q167. Why can ORDER BY use a SELECT alias when WHERE cannot?

Q168. Walk through this query in execution order and describe the result. Is the output deterministic?

```
SELECT customer_id, COUNT(*) AS n
FROM customer_purchases
WHERE product_id = 2
GROUP BY customer_id
HAVING n >= 5
ORDER BY n DESC
LIMIT 2;
```

Q169. "Average of the 10 largest purchase lines". Why is this query wrong? Fix it.

```
SELECT AVG(quantity * cost_to_customer_per_qty)
FROM customer_purchases
ORDER BY quantity * cost_to_customer_per_qty DESC
LIMIT 10;
```

Q170. What does each of these return?

```
SELECT COUNT(*) FROM customer_purchases WHERE 1 = 0;
SELECT COUNT(*) FROM customer_purchases WHERE 1 = 0 GROUP BY customer_id;
```

Section 15: Subqueries

Q171. Find the purchase lines whose value is above the average line value. How many are there?

Q172. Find the customers who have **never** made a purchase, first with NOT IN and then with NOT EXISTS. What does the result tell you about this extract?

Q173. Without a JOIN, show the **names** of customers who spent more than $200.

Q174. Find the names of the products that have been sold at least once.

Q175. For each purchase line, show the customer's **own** average line value next to it, plus the difference. Use a correlated subquery.

Q176. Find the purchases where the price paid was lower than that day's `original_price` in `vendor_inventory`.

Q177. For each product, find the market date with the highest revenue.

Q178. Find the customer with the highest total spend **without** LIMIT, so that ties are returned too.

Q179. Find the market dates whose revenue was above the average **daily** revenue.

Q180. For product 1, compute the sell-through % on each market date: quantity sold ÷ quantity brought. Get the quantity brought with a correlated subquery on `vendor_inventory`.

Q181. Using EXISTS, find the vendors with inventory records for product 4 in 2020.

Q182. Show each customer's share of total revenue as a percentage.

Q183. Find the market date(s) with the most distinct customers, including ties.

Q184. Find the customer with the **second highest** total spend without LIMIT or OFFSET.

Section 16: Statistics

Q185. For purchase quantity, compute the mean, population standard deviation, sample standard deviation and variance. What is the difference between `STD()`, `STDDEV_POP()` and `STDDEV_SAMP()`?

Q186. Find the **median** purchase quantity using subqueries. Make sure it works for both odd and even row counts and for ties.

Q187. Find the **mode** of purchase quantity, including all values that tie.

Q188. Using z-scores, find the outlier purchase lines whose value is more than 2 standard deviations above the mean.

Q189. For each product in `vendor_inventory`, compute the coefficient of variation (stddev ÷ mean) of the daily quantity brought. Which product has the least predictable supply?

Q190. For each season, show the number of recorded days, the mean, the min, the max, the range and the mid-range of `market_max_temp`.

Q191. Compute the Pearson correlation between `market_min_temp` and `market_max_temp` using only aggregate functions.

Q192. Find the **top 10%** of purchase lines by value using a correlated subquery.

Q193. For each market date, show that day's revenue, the **previous market day's** revenue and the % change. Use correlated subqueries.

Q194. Compute a running (cumulative) total of revenue by market date using a correlated subquery.

Section 17: Interview Logic Scenarios

Q195. Find the customers who shopped on **both** the Wednesday and the Saturday of the same market week at least once. How many such weeks did each customer have?

Q196. What percentage of customers are **repeat** customers (purchases on more than one market date)? And what percentage made purchases in both 2019 and 2020?

Q197. Find the customers whose 2020 spend was higher than their 2019 spend. Show both amounts and the growth %.

Q198. Find the market dates when vendor 7 brought Habanero Peppers (product 1) but **sold none**.

Q199. Build a year-over-year table for July, August and September: one row per month, with columns for 2019 revenue, 2020 revenue and YoY growth %.

Q200. Build a one-row **executive summary** using scalar subqueries only: total revenue, total customers, best customer id, best market date (by revenue) and busiest hour (by line count).